Informe Técnico Consolidado - NLP Capstone

Clasificación de noticias con TF-IDF frente a Transformer + LoRA

María Lorena Gómez
CODERHOUSE | DATASCIENCE III - Comisión #102200

Índice

Resumen ejecutivo 3
1. Definición del problema 4
1.1 Contexto y objetivo 4
1.2 Dataset: AG News 4
1.3 Hipótesis y ejes de análisis 4
1.4 Hilo conductor: de las cuatro pre-entregas al capstone 5
1.5 Protocolo experimental y reglas de rigor 5
1.6 Métrica principal: F1 ponderado (weighted) 5
2. Pipeline de preprocesamiento y análisis exploratorio 6
2.1 Diagnóstico: ruido de codificación y firmas de agencia 6
2.2 Pipeline: dos ramas con un tronco común 6
2.3 Análisis exploratorio (EDA técnico) 7
2.4 Tokenización: scikit-learn frente a Hugging Face 10
2.5 Detalles de reproducibilidad de la tokenización 10
3. Desarrollo del baseline (Módulo 3): TF-IDF + clasificador 12
3.1 Por qué empezar con un baseline clásico 12
3.2 Diseño experimental 12
3.3 Resultados de la exploración de hiperparámetros 12
3.4 Modelo final y resultados en el test 13
3.5 Interpretabilidad del baseline: términos más influyentes 14
3.6 Análisis de errores y categorías difíciles 14
3.7 Justificación de la elección del modelo 15
4. Desarrollo de la solución avanzada (Módulo 4): DistilBERT + LoRA 16
4.1 Qué se reutiliza del Módulo 1 16
4.2 Selección del modelo base 16
4.3 Arquitectura del modelo 16
4.4 Configuración de LoRA (PEFT) 16
4.5 Ciclo de entrenamiento 18
4.6 Artefacto resultante 19
5. Evaluación comparativa 20
5.1 Protocolo de comparación (manzanas con manzanas) 20
5.2 Tabla comparativa de métricas 20
5.3 Desempeño por clase 20
5.4 Matrices de confusión comparadas 21
5.5 ¿La diferencia es estadísticamente significativa? 22
5.6 Complejidad léxica y cobertura del vocabulario 23
5.7 Semántica de secuencias e interpretabilidad: qué capturó B que A ignoró 24
5.8 Análisis de eficiencia: tiempo de entrenamiento frente a desempeño 25

5.9 Respuesta a la pregunta central: el porqué desde los tres ejes 26
6. Conclusiones, limitaciones y mejoras futuras 27
6.1 Hallazgos principales 27
6.2 Recomendación al negocio 27
6.3 Limitaciones del sistema y de la evidencia 27
6.4 Líneas de mejora futuras 28
6.5 Referencias 28
Anexo A. Pre-entrega 1 (Módulo 1): pipeline de entrenamiento, validación y clasificador base 30
A.1 Entorno y reproducibilidad 30
A.2 Datos y arquitectura 30
A.3 Ciclo de entrenamiento y validación 30
A.4 Resultados por época 31
A.5 Interpretación de la curva de pérdida 31
A.6 README de la pre-entrega 1 (consolidado) 32
Anexo B. Matriz de cumplimiento de rúbricas 33
Anexo C. Guía de reproducción, estructura del repositorio y versiones 35
C.1 Estructura del repositorio 35
C.2 Ejecución 35
C.3 Versiones registradas 35
C.4 Contenido sugerido de requirements.txt 36
Anexo D. Tablas completas y archivos de configuración 37
D.1 Grilla completa de TF-IDF (validación interna, 18 combinaciones) 37
D.2 Matrices de confusión numéricas (test) 37
D.3 adapter_config.json (resumen) 37
D.4 Salidas de predicción 38
Anexo E. Glosario de términos técnicos 39

Resumen ejecutivo

Este informe consolida de punta a punta el proyecto integrador de **Data Science III (NLP y Deep Learning)**: desde el pipeline de entrenamiento en PyTorch (Módulo 1) y el preprocesamiento y EDA del corpus (Módulo 2), pasando por el baseline clásico TF-IDF + clasificador (Módulo 3) y el fine-tuning eficiente de un Transformer con LoRA (Módulo 4), hasta la comparación técnica final (Módulo 5). Todo el trabajo usa un único corpus, **AG News** (8.000 noticias de entrenamiento y 2.000 de test, 4 clases perfectamente balanceadas), y ambos modelos se evalúan sobre exactamente el mismo **`ag_news_test.csv`**.

Tabla 1. Resultado principal (test, 2.000 noticias)

Métrica	A: TF-IDF + MultinomialNB	B: DistilBERT + LoRA (r=8)	Diferencia B − A
F1 weighted (= macro)	0.8960	0.9082	+0.0122
Precision weighted	0.8964	0.9091	+0.0127
Recall weighted	0.8960	0.9080	+0.0120
Accuracy	0.8960	0.9080	+0.0120
Parámetros entrenados	25.400 features (NB)	741.124 (1.09% del modelo)	—
Tiempo de entrenamiento	0.16 s (CPU)	219.4 s (GPU Apple MPS)	≈ 1.371× más
Inferencia sobre el test	< 0,01 s	8.7 s	—

Conclusión en una línea. El Transformer con LoRA mejora el F1 en **+0.0122** (IC 95% pareado por bootstrap: [+0.0015, +0.0232]; prueba de McNemar $\chi^2 = 4.34$, $p = 0.037$), una mejora **estadísticamente detectable pero modesta**, a cambio de entrenar ≈ 1.371 veces más lento y de necesitar GPU. En un problema de clasificación temática donde el vocabulario distintivo ya es muy informativo, el baseline clásico es una opción muy competitiva; el Transformer se justifica cuando cada punto de F1 tiene valor de negocio.

Además de reunir el material de las cuatro pre-entregas, este informe agrega análisis nuevos, calculados a partir de los resultados reales guardados por el proyecto: significancia estadística (McNemar y bootstrap), análisis de cobertura léxica, análisis por longitud de documento, explicación de errores del baseline término por término y una **verificación de reproducibilidad** (al reentrenar el baseline se obtienen predicciones idénticas al 100% a las guardadas).

> **Cómo leer este documento.** Las secciones 1 a 6 son el Informe Técnico Consolidado (estructura pedida por la consigna del TP final). Los anexos A a E conservan el material propio de cada pre-entrega (en especial el Módulo 1), la matriz de cumplimiento de rúbricas, la guía de reproducción, tablas completas y un glosario.

1. Definición del problema

1.1 Contexto y objetivo

Los medios digitales reciben miles de titulares y notas por hora que deben **enrutarse automáticamente** a la sección correcta (Mundo, Deportes, Negocios, Ciencia/Tecnología). Es un caso de negocio típico de NLP: clasificación temática de texto corto, con alto volumen y necesidad de latencia baja. El objetivo de investigación del proyecto es responder una pregunta concreta de ingeniería de ML:

> **¿Cuánto aporta, y a qué costo, un Transformer adaptado con LoRA frente a un baseline clásico bien construido (TF-IDF + clasificador lineal/bayesiano), evaluados sobre el mismo conjunto de prueba?**

1.2 Dataset: AG News

Se utiliza **AG News** (Zhang, Zhao y LeCun, 2015), corpus de noticias en inglés construido a partir del buscador de noticias de AG. La cátedra entrega una muestra balanceada y recortada, con los splits ya separados. Se eligió AG News (y no BBC News) por ser liviano y permitir fine-tuning rápido; se mantuvo **el mismo corpus en los cuatro módulos**, como exige la consigna.

Tabla 2. Características del dataset

Atributo	Valor
Idioma / tarea	Inglés / clasificación multiclase de noticias por tema
Archivos	`ag_news_train.csv` (8.000 filas), `ag_news_test.csv` (2.000 filas); columnas `text` y `label`
Clases	4: `Business`, `Sci_Tech`, `Sports`, `World`
Balance	Train: 2.000 por clase · Test: 500 por clase (perfectamente balanceado)
Formato del texto	Titular + cuerpo breve, p. ej. “Red Hat Appoints New CFO. September 2, 2004 -- …”
Longitud	Mediana ≈ 37 tokens lematizados; percentil 95 = 52 (p. ej. 71 subpalabras WordPiece)
Ruido de codificación	2.639 documentos de train (33,0%) con entidades/tags/URLs; 1.688 (21,1%) con firmas de agencia
Valores nulos	Ninguno (según el README de la cátedra)

1.3 Hipótesis y ejes de análisis

La consigna pide explicar no solo **qué** modelo funciona mejor sino **por qué**, desde tres perspectivas. Cada una se traduce en una hipótesis contrastable con los datos del proyecto (se discuten en la sección 5.9):

Eje	Pregunta	Hipótesis de partida	Evidencia usada
Complejidad léxica	¿El vocabulario del test es muy distinto del de train?	El baseline sufre con términos raros y variantes de superficie; el Transformer, con sub-palabras, es más robusto.	Tasa de tokens fuera de vocabulario (OOV), 17.719 tipos en train, hápax
Cobertura del vocabulario	¿Qué ve cada representación de un mismo texto?	TF-IDF descarta lo que no está en su vocabulario; WordPiece cubre todo texto con unidades conocidas.	OOV por grupo de aciertos; parámetros de TF-IDF y WordPiece
Semántica de secuencias	¿El orden y el contexto cambian la clase?	El Transformer desambigua palabras polisémicas y metáforas; la bolsa de palabras no.	Errores corregidos por B, contribución de términos del Naive Bayes, matrices de confusión

1.4 Hilo conductor: de las cuatro pre-entregas al capstone

El proyecto se construyó de manera incremental. Cada módulo deja un artefacto que el siguiente reutiliza; el informe final no es una suma de partes sino una cadena en la que **el mismo corpus, la misma regla de oro (test intocable) y el mismo orden de clases** conectan todas las etapas.

Módulo	Entregable	Qué aporta al informe final
M1 · Redes neuronales básicas	Pipeline PyTorch: dispositivo, semilla, MLP, ciclo de entrenamiento/validación con Adam	El "motor" de entrenamiento (forward → loss → zero_grad → backward → step) y el diagnóstico de sobreajuste que se reutiliza en M4 (Anexo A)
M2 · NLP I	Corpus preprocesado + EDA técnico (PDF)	Función de limpieza y lematización, p95 de longitud, n-gramas, balance de clases (sección 2)
M3 · NLP II	Clasificador supervisado TF-IDF (notebook + README)	Baseline A: grilla de hiperparámetros, métricas y errores (sección 3)
M4 · Deep Learning	Fine-tuning con LoRA (PDF)	Solución avanzada B: DistilBERT + LoRA, parámetros y tiempos (sección 4)
M5 · Capstone	Informe técnico consolidado	Comparación A vs B, explicación del porqué, conclusiones (secciones 5 y 6)

1.5 Protocolo experimental y reglas de rigor

- **El test nunca se usa para entrenar ni para elegir hiperparámetros.** Las decisiones se tomaron en validaciones internas: 20% estratificado del train en M3 (6.400 / 1.600) y 10% estratificado en M4 (7.200 / 800).
- **Sin fuga de datos:** `TfidfVectorizer` se ajusta (`fit`) solo con train y se aplica `transform` al test. La lematización y la limpieza no tienen parámetros aprendidos, por lo que aplicarlas igual a train y test no introduce fuga.
- **Mismo test para ambos modelos**, con el mismo orden de clases (`Business, Sci_Tech, Sports, World`) y las mismas 2.000 noticias.
- **Reproducibilidad:** semilla 42 en todos los pasos; versiones y parámetros documentados (Anexo C).

1.6 Métrica principal: F1 ponderado (weighted)

La consigna advierte contra defender el proyecto solo con Accuracy y pide priorizar el **F1 ponderado**. En AG News las clases están perfectamente balanceadas (500 por clase en test), por lo que **F1 macro y F1 weighted coinciden** (y el Accuracy coincide con el Recall); aun así se reporta F1 weighted como métrica principal, junto con Precision, Recall, F1 por clase y matrices de confusión. Se aclara explícitamente que esta equivalencia es una propiedad del dataset recortado y **no se extrapola a escenarios reales desbalanceados** (sección 6.3).

2. Pipeline de preprocesamiento y análisis exploratorio

2.1 Diagnóstico: ruido de codificación y firmas de agencia

El primer hallazgo del EDA fue que AG News, aunque se describe como "limpio", contiene artefactos de codificación que contaminan el vocabulario (`amp`, `lt`, `39`, `quot`) y firmas repetidas que duplican n-gramas sin aportar tema. Ejemplos reales del corpus:

Artefacto	Ejemplo en el corpus	Tratamiento
Entidades HTML escapadas	`<b>`	`html.unescape` y luego eliminación de tags
Entidades numéricas sin "&"	`Macy #39;s` (apóstrofo), `#36;500,000` ($), `#151;` (guion)	Regex `#(\d+);` → carácter correspondiente (ASCII / cp1252)
Entidades nombradas sin "&"	`quot;`, `amp;`, `nbsp;`, `lt;`, `gt;`	Reemplazo por su carácter o espacio
Tags HTML	`<b>…</b>`, ` `	Regex `<[^>]*>` → espacio
URLs	`http://…`, `www.…`	Regex `https?://\S+|\bwww\.?\S*` → espacio
Firmas de agencia	`(AP) AP -`, `(Reuters) Reuters -`, `Quote, Profile, Research`	Regex específica; evita bigramas espurios como "ap ap"

Resultado medido en train: **2.639 documentos con ruido antes de limpiar y 0 después**; 1.688 documentos con firmas de agencia eliminadas.

2.2 Pipeline: dos ramas con un tronco común

El tronco común es la **limpieza estructural** (`clean_text`). A partir de ahí el pipeline se bifurca de forma deliberada, porque cada familia de modelos se beneficia de una representación distinta:

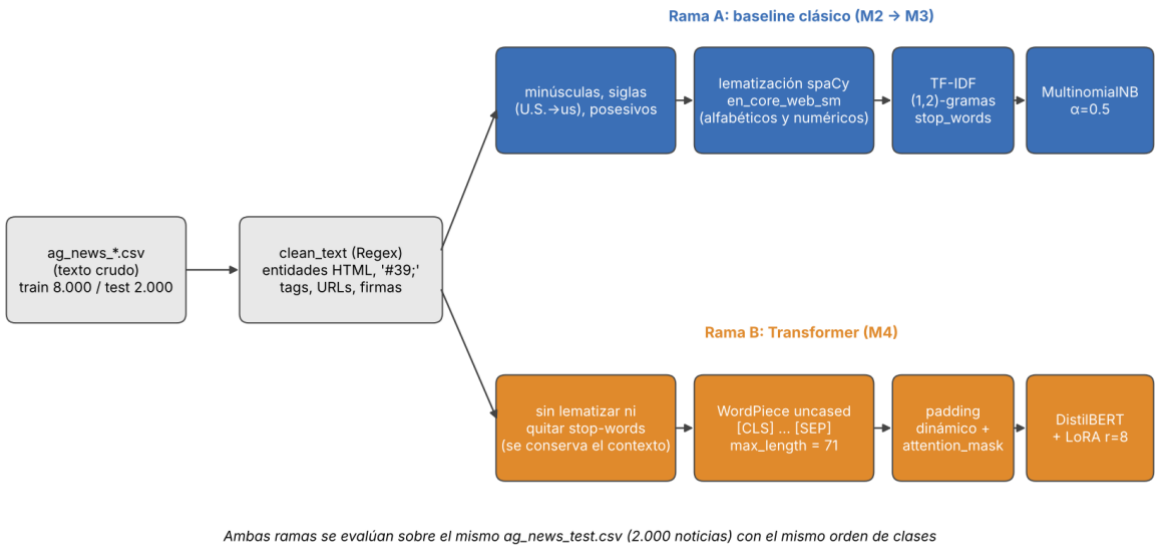


Figura 1. Pipeline de preprocesamiento: tronco común y dos ramas (baseline y Transformer).

Tabla 3. Qué se aplica a cada rama y por qué

Paso	Rama A (TF-IDF + NB)	Rama B (DistilBERT + LoRA)	Justificación
Limpieza Regex (entidades, tags, URLs, firmas)	Sí	Sí	Elimina ruido mecánico que ninguno de los modelos debería aprender
Minúsculas	Sí (explícito)	Sí (el tokenizer es *uncased*)	Reduce variantes de superficie
Siglas (U.S. → us) y posesivos ('s)	Sí	No	Unifica tokens para la bolsa de palabras

Paso	Rama A (TF-IDF + NB)	Rama B (DistilBERT + LoRA)	Justificación
Lematización (spaCy `en_core_web_sm`)	Sí: conserva tokens alfabéticos y numéricos	No	Reduce la dimensionalidad del vocabulario; el Transformer aprovecha formas flexionadas
Eliminación de stop-words	Sí, dentro de `TfidfVectorizer` (`stop_words='english'`)	No	Los Transformers usan el orden y las palabras funcionales como contexto
Puntuación	Descartada	Conservada	La puntuación ayuda a segmentar (p. ej. el punto tras el titular)
Tokenización	Palabras/lemas + n-gramas (1,2)	WordPiece (sub-palabras)	Ver 2.4

> **Por qué la diferencia es deliberada.** Las representaciones esparsas (bolsa de palabras) mejoran cuando se reduce la variabilidad léxica: lematizar y quitar stop-words concentra la señal en menos columnas. La auto-atención, en cambio, modela relaciones entre *todos* los tokens, de modo que borrar palabras funcionales o formas verbales elimina información contextual. Esta separación mantiene un hilo conductor único (misma limpieza estructural) pero respeta la naturaleza de cada modelo.

2.3 Análisis exploratorio (EDA técnico)

Todas las estadísticas siguientes corresponden a la corrida del pipeline con **spaCy `en_core_web_sm`** sobre el train (8.000 documentos); el EDA entregado en el Módulo 2 se había generado con un lematizador de reglas de respaldo y presenta diferencias menores (Anexo D).

Longitud de documentos y percentil 95

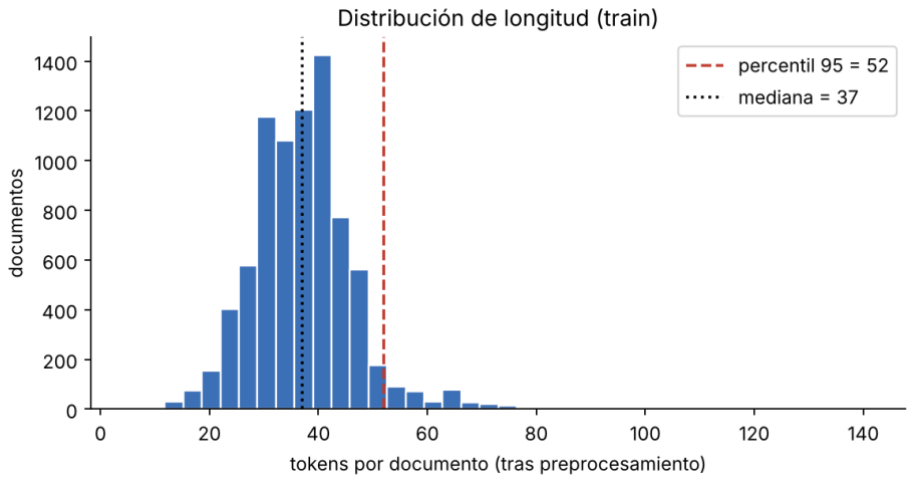


Figura 2. Histograma de tokens por documento (train, tras el preprocesamiento).

Los textos son cortos: **media 37.2, mediana 37, mínimo 5, máximo 141 tokens**. El **percentil 95 es 52 tokens** (valor de referencia de `max_len` para la rama de palabras). En la rama del Transformer el percentil 95 se recalculó en sub-palabras: **71** (más alto porque WordPiece parte palabras raras y cuenta tokens especiales); se usó `max_length = 71`, por lo que por construcción solo ≈ 5% de las noticias se truncan.

Distribución de clases

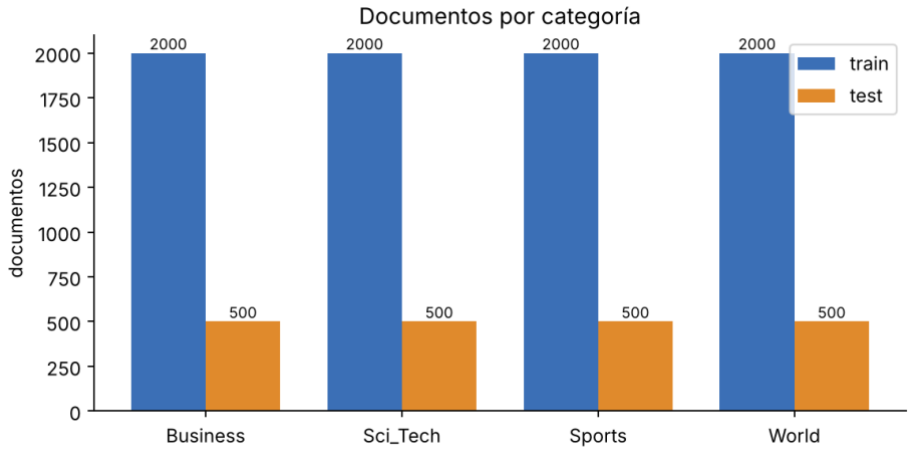


Figura 3. Documentos por categoría en train y test.

Las cuatro clases están **perfectamente balanceadas** (2.000 por clase en train, 500 en test). Consecuencias para M3 y M4: no se requiere `class_weight` ni remuestreo, y F1 macro = F1 weighted.

Bi-gramas y tri-gramas más frecuentes

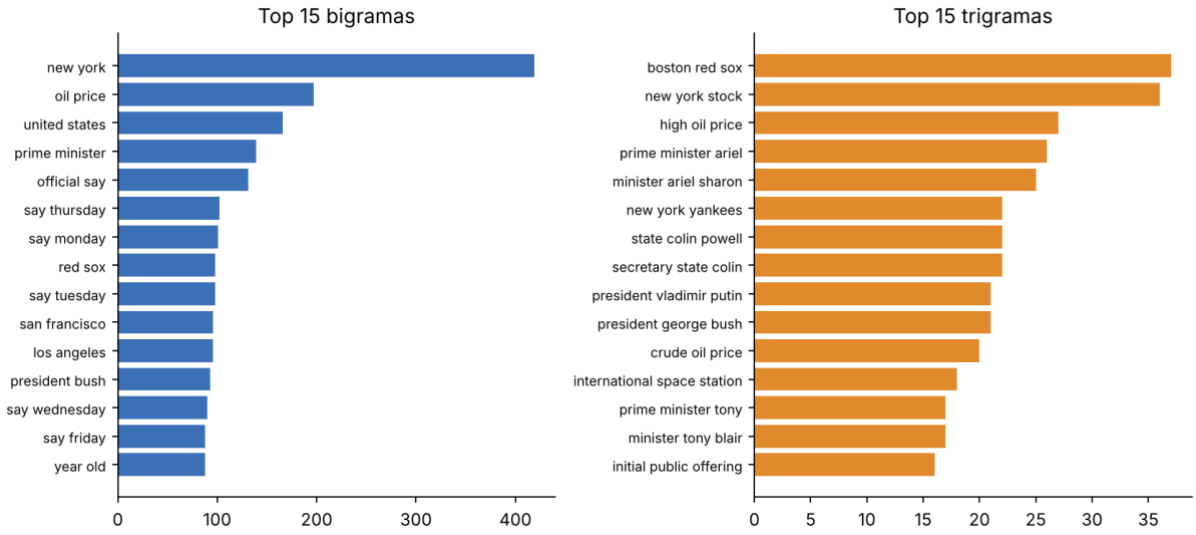


Figura 4. Top 15 bi-gramas y tri-gramas (scikit-learn, sin stop-words, texto lematizado).

Tabla 4. Top 20 bi-gramas y tri-gramas (frecuencia en train)

#	Bi-grama	Frec.	Tri-grama	Frec.
1	new york	419	boston red sox	37
2	oil price	197	new york stock	36
3	united states	166	high oil price	27
4	prime minister	139	prime minister ariel	26
5	official say	131	minister ariel sharon	25
6	say thursday	102	new york yankees	22
7	say monday	101	state colin powell	22
8	red sox	98	secretary state colin	22
9	say tuesday	98	president vladimir putin	21
10	san francisco	96	president george bush	21
11	los angeles	96	crude oil price	20

#	Bi-grama	Frec.	Tri-grama	Frec.
12	president bush	93	international space station	18
13	say wednesday	90	prime minister tony	17
14	say friday	88	minister tony blair	17
15	year old	88	initial public offering	16
16	sport network	74	los angeles dodger	16
17	mobile phone	72	security exchange commission	16
18	wall street	71	osama bin laden	15
19	microsoft corp	70	wal mart stores	15
20	world cup	69	oil price rise	15

Interpretación. Los n-gramas son entidades y expresiones típicas de prensa (organizaciones, cargos, eventos, deportes) y **no contienen tags HTML ni fragmentos como `amp`, `lt` o `39`**, lo que valida la limpieza. Los bigramas reflejan los cuatro temas: *oil price*, *wall street*, *microsoft corp* (negocios/tecnología), *prime minister*, *president bush* (mundo), *red sox*, *world cup* (deportes). Aparecen además patrones de "día de la semana" (*say thursday*, *say monday*), propios del estilo de agencia, que son candidatos a ruido para un clasificador temático. Es notable que los lemas ("say", "official say") reemplazan a "said" del texto original.

Palabras más frecuentes y stop-words

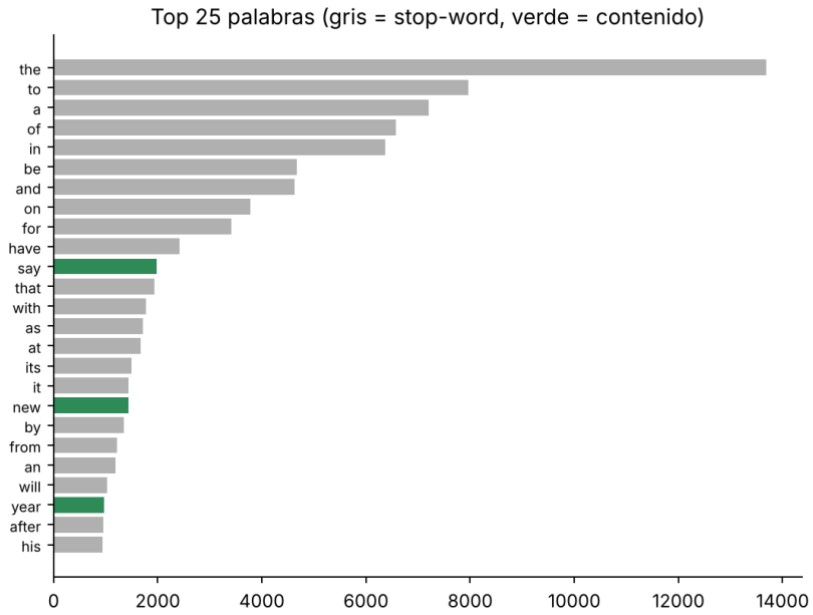


Figura 5. Top 25 palabras más frecuentes (gris: stop-word; verde: palabra de contenido).

Tabla 5. Top 50 palabras (texto lematizado, sin quitar stop-words)

#	Palabra	Frec.	Stop?	#	Palabra	Frec.	Stop?
1	the	13687	sí	26	us	936	sí
2	to	7962	sí	27	over	803	sí
3	a	7200	sí	28	their	733	sí
4	of	6577	sí	29	up	730	sí
5	in	6365	sí	30	he	723	sí
6	be	4678	sí	31	two	675	sí
7	and	4633	sí	32	first	672	sí
8	on	3772	sí	33	company	651	

#	Palabra	Frec.	Stop?	#	Palabra	Frec.	Stop?
9	for	3406	sí	34	world	609	
10	have	2422	sí	35	but	598	sí
11	say	1975		36	win	595	
12	that	1932	sí	37	one	581	sí
13	with	1771	sí	38	more	578	sí
14	as	1717	sí	39	they	577	sí
15	at	1668	sí	40	game	554	
16	its	1501	sí	41	report	549	
17	it	1437	sí	42	out	545	sí
18	new	1432		43	this	543	sí
19	by	1356	sí	44	we	540	sí
20	from	1221	sí	45	not	530	sí
21	an	1194	sí	46	oil	519	
22	will	1027	sí	47	monday	518	
23	year	967		48	wednesday	505	
24	after	947	sí	49	thursday	502	
25	his	940	sí	50	tuesday	495	

De las 50 palabras más frecuentes, **37 son stop-words** (lista `ENGLISH_STOP_WORDS` de scikit-learn). El vocabulario lematizado tiene **17.719 tipos, de los cuales 6.968 (39.3%) aparecen una sola vez** (cola larga de Zipf). Esto tiene dos consecuencias de diseño: (i) en la rama clásica conviene filtrar stop-words y usar `min_df=2` para descartar los hápax; (ii) en la rama Transformer las palabras raras se descomponen en sub-palabras conocidas, evitando el problema de vocabulario cerrado.

2.4 Tokenización: scikit-learn frente a Hugging Face

Aspecto	Scikit-Learn (`TfidfVectorizer`)	Hugging Face (WordPiece de `distilbert-base-uncased`)
Unidad de token	Palabra/lema y n-gramas de palabras (1,2)	Sub-palabra (fragmentos de palabra; los prefijos internos llevan `##`)
Vocabulario	Aprendido del train: 25.400 términos (1-2 gramas, `min_df=2`)	Fijo y preentrenado (≈ 30.000 tipos); no se aprende del corpus
Palabras nuevas (OOV)	Se ignoran en la inferencia	Se descomponen en fragmentos o caracteres conocidos; prácticamente no hay OOV
Orden de las palabras	Se pierde (bolsa de palabras); solo se recupera localmente con bigramas	Se preserva: embeddings posicionales + auto-atención
Representación	Vector esparso de pesos TF-IDF (norma L2)	Embeddings densos contextuales de 768 dimensiones por token
Normalización previa	Lematización + stop-words (nuestro pipeline)	Solo limpieza estructural; el tokenizer pasa a minúsculas
Longitud	Variable; sin límite	Truncado a 71 tokens; padding dinámico por lote

2.5 Detalles de reproducibilidad de la tokenización

La consigna advierte que omitir special tokens y longitud de padding impide replicar el modelo. Los parámetros exactos usados son:

Parámetro	Valor
Modelo / tokenizer	`distilbert-base-uncased` (`AutoTokenizer`, WordPiece, minúsculas)
Tokens especiales	`[CLS]` al inicio, `[SEP]` al final, `[PAD]` para relleno, `[UNK]` y `[MASK]` (no usados en la práctica)
Truncation	`truncation=True`, `max_length = 71` (= percentil 95 de longitud en sub-palabras, tope 128)
Padding	Dinámico por lote con `DataCollatorWithPadding` (se rellena hasta el documento más largo del lote); `attention_mask` marca tokens reales
Texto de entrada	Solo `clean_text` (limpieza estructural); sin lematizar ni quitar stop-words
Batch size	32 (train) y 128 (evaluación)
Etiquetas	Enteros por orden alfabético: Business=0, Sci_Tech=1, Sports=2, World=3 (`id2label` / `label2id` guardados en el modelo)
TF-IDF (rama A)	`sublinear_tf=True`, `min_df=2`, `stop_words='english'`, `ngram_range=(1,2)`, `max_features=None`, norma L2 por defecto

3. Desarrollo del baseline (Módulo 3): TF-IDF + clasificador

3.1 Por qué empezar con un baseline clásico

Antes de desplegar un Transformer, una práctica profesional es fijar un punto de referencia barato, rápido e interpretable. TF-IDF pondera cada término por su frecuencia en el documento (TF) y su rareza en el corpus (IDF, con logaritmo para amortiguar la ley de Zipf), y normaliza cada vector con norma L2 para eliminar el sesgo por longitud. El resultado es una matriz esparsa que scikit-learn procesa sin densificar. Este baseline es el listón que la solución avanzada debe superar **y justificar**.

3.2 Diseño experimental

Elemento	Configuración
Entrada	Texto preprocesado de la rama A (limpieza Regex + minúsculas + lematización spaCy)
Validación	20% estratificado del train (6.400 entrenamiento interno / 1.600 validación), semilla 42. El test no interviene
Vectorizador	`TfidfVectorizer(sublinear_tf=True, min_df=2, stop_words='english')`; **se exploran** `max_features` ∈ {5.000, 20.000, sin límite} y `ngram_range` ∈ {(1,1), (1,2)}
Clasificadores	`MultinomialNB(alpha=0.5)`, `LogisticRegression(C=10, max_iter=1000)`, `LinearSVC(C=0.5)`
Grilla	3 × 2 × 3 = 18 combinaciones; criterio: F1 macro en validación
Modelo final	Mejor combinación, reentrenada sobre **todo** el train (`fit_transform` solo en train) y evaluada **una única vez** en el test

3.3 Resultados de la exploración de hiperparámetros

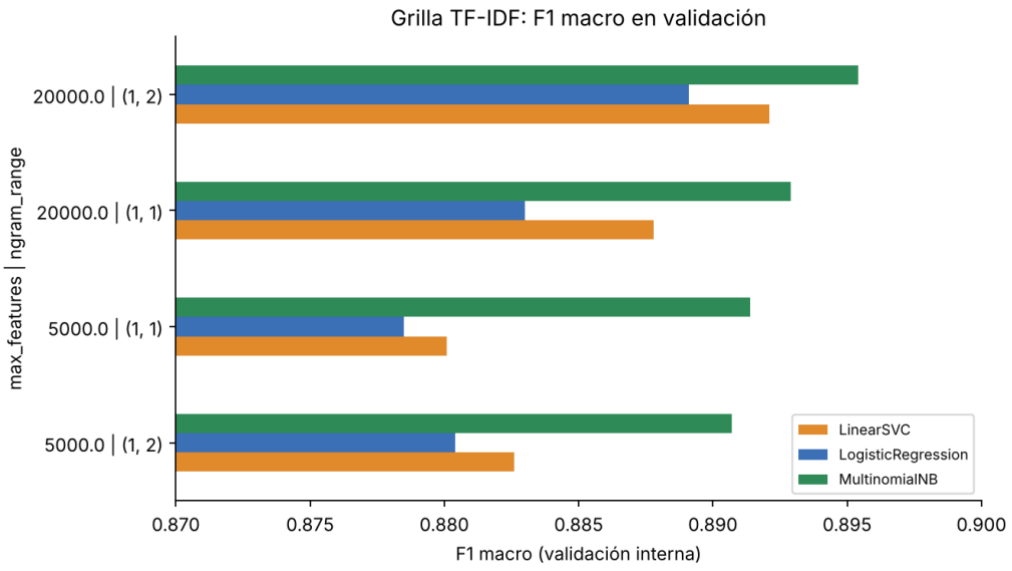


Figura 6. F1 macro en validación para las 18 combinaciones (ordenadas por mejor modelo por configuración).

Tabla 6. Las 8 mejores configuraciones en validación (tabla completa en el Anexo E)

#	max_features	ngram_range	Modelo	n_features	F1 macro val.
1	sin límite	"(1	2)"	MultinomialNB	20507
2	20000	"(1	2)"	MultinomialNB	20000
3	sin límite	"(1	2)"	LinearSVC	20507
4	20000	"(1	1)"	MultinomialNB	8253
5	sin límite	"(1	1)"	MultinomialNB	8253

#	max_features	ngram_range	Modelo	n_features	F1 macro val.
6	20000	"(1	2)"	LinearSVC	20000
7	5000	"(1	1)"	MultinomialNB	5000
8	5000	"(1	2)"	MultinomialNB	5000

Lectura de la grilla. (i) **Naive Bayes domina**: ocupa 6 de los 8 primeros lugares, aun con su supuesto de independencia condicional, algo coherente con textos cortos y vocabulario muy discriminativo por tema. (ii) **Los bigramas ayudan** (`(1,2)` supera a `(1,1)` en NB: 0,8960 frente a 0,8929) porque capturan contexto local como *new york*, *oil price* o *red sox*. (iii) **Limitar `max_features` a 5.000 cuesta ≈ 0,5 a 1,1 puntos de F1** y no aporta un beneficio visible de regularización en este corpus; (iv) el rango total entre la peor (0,8785) y la mejor (0,8960) configuración es de solo 1,75 puntos.

> **Cautela estadística.** Con 1.600 documentos de validación, el error estándar de una exactitud ≈ 0,90 es ≈ 0,75 puntos porcentuales. Por lo tanto, las primeras 3 o 4 configuraciones son **estadísticamente indistinguibles** entre sí; la elección de `MultinomialNB (1,2) sin límite` es razonable (mejor valor puntual, modelo más simple y más rápido), pero no debe leerse como una superioridad demostrada sobre `LinearSVC`.

3.4 Modelo final y resultados en el test

Modelo elegido: **TF-IDF (1,2)-gramas sin límite de `max_features` + MultinomialNB (α = 0,5)**. Al reentrenar con todo el train el vocabulario crece a **25.400 características** (frente a 20.507 en el entrenamiento interno de 6.400 documentos, porque hay más documentos que alcanzan `min_df=2`). El entrenamiento completo (vectorizador + clasificador) tomó **0.16 s en CPU**; la inferencia sobre las 2.000 noticias es despreciable.

Tabla 7. classification_report del baseline sobre el test

Clase	Precision	Recall	F1	Soporte
Business	0.8395	0.8680	0.8535	500
Sci_Tech	0.8803	0.8680	0.8741	500
Sports	0.9490	0.9680	0.9584	500
World	0.9167	0.8800	0.8980	500
accuracy			0.8960	2000
macro avg	0.8964	0.8960	0.8960	2000
weighted avg	0.8964	0.8960	0.8960	2000

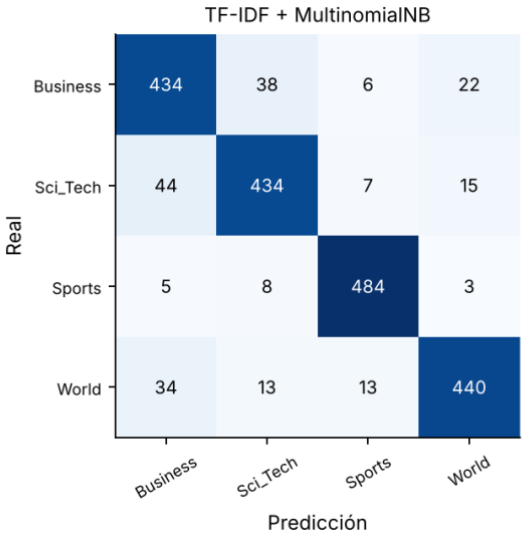


Figura 7. Matriz de confusión del baseline (test).

3.5 Interpretabilidad del baseline: términos más influyentes

Una ventaja del baseline es que se puede inspeccionar. Con Naive Bayes se consultan las log-probabilidades de cada término por clase (`feature_log_prob_`):

Clase	Términos con mayor peso (lemas)
Business	say, oil, price, company, new, profit, stock, rise, percent, share
Sci_Tech	microsoft, new, say, software, internet, company, service, computer, space, use
Sports	win, game, season, team, night, cup, victory, league, year, new
World	say, iraq, kill, president, minister, election, leader, official, government, people

Los términos son semánticamente razonables (*iraq, minister, election* para World; *win, game, season* para Sports; *microsoft, software, internet* para Sci_Tech; *oil, price, profit, stock* para Business). También revelan el problema de fondo: términos como *say*, *new* y *company* aparecen entre los más influyentes de **varias clases a la vez**, porque son frecuentes en toda la prensa y el modelo no sabe discriminar por contexto.

3.6 Análisis de errores y categorías difíciles

Tabla 8. Confusiones más frecuentes del baseline (real → predicho)

Real → Predicho	Casos	% de la clase real
Sci_Tech → Business	44	8.8%
Business → Sci_Tech	38	7.6%
World → Business	34	6.8%
Business → World	22	4.4%
Sci_Tech → World	15	3.0%
World → Sci_Tech	13	2.6%

La categoría **más difícil es Business** (F1 = 0.8535) y la **más fácil es Sports** (F1 = 0.9584), cuyo vocabulario (*game, season, team, win*) es muy específico. El **par más confundido es Sci_Tech ↔ Business** (44 + 38 = 82 casos): ambas clases comparten términos de empresas, mercados y productos (*company, new, say, software*). Esto es esperable en una bolsa de palabras, que no capta orden ni contexto, y es justamente la limitación que se pretende atacar con el Transformer.

3.7 Justificación de la elección del modelo

- **Naive Bayes multinomial** obtuvo el mejor F1 macro de validación (0,8960), es el más rápido (entrena en décimas de segundo) y trabaja directamente sobre la matriz esparsa; suele rendir bien en clasificación temática de textos cortos.
- **Regresión logística** (F1 val. máx. 0,8891) ofrece probabilidades calibradas e interpretabilidad por coeficientes, pero fue inferior y más lenta (≈ 0,3–0,4 s por ajuste frente a 0,01 s de NB).
- **SVM lineal** (F1 val. máx. 0,8934) es un estándar robusto en alta dimensión y queda estadísticamente empatada con NB; sería la alternativa natural si se priorizara el margen sobre la velocidad.

Con estas métricas, el baseline cumple el criterio de la consigna: un baseline coherente (F1 ≈ 0,90 en AG News coincide con la referencia indicada por la cátedra para LogisticRegression) contra el cual comparar.

4. Desarrollo de la solución avanzada (Módulo 4): DistilBERT + LoRA

4.1 Qué se reutiliza del Módulo 1

El Trainer de Hugging Face encapsula el mismo ciclo que se implementó explícitamente en el Módulo 1 con PyTorch: **forward pass → cálculo de la pérdida (entropía cruzada) → `zero_grad` → `backward` (autograd aplica la regla de la cadena, es decir, *backpropagation*) → `step` del optimizador**. Lo aprendido en M1 (detección de dispositivo cuda/mps/cpu, semilla fija, validación por época para detectar sobreajuste, curvas de pérdida) se aplica aquí tal cual. En M1 el MLP sobre TF-IDF mostró sobreajuste a partir de la época 3 (Anexo A), lo que motivó usar **validación por época, selección del mejor checkpoint por F1 macro y pocas épocas (3)** en M4.

4.2 Selección del modelo base

Candidato	Parámetros (aprox.)	Capas / cabezas	Idioma	Evaluación
`bert-base-uncased`	≈ 110 M	12 / 12	Inglés	Referencia de calidad; más lento y pesado
`roberta-base`	≈ 125 M	12 / 12	Inglés	Suele rendir algo más; pero es el más costoso de los tres
`distilbert-base-uncased`	**≈ 67 M** (66.956.548 con cabeza de 4 clases)	**6 / 12**	Inglés	**Elegido**: conserva ≈ 97% del desempeño de BERT con ≈ 40% menos parámetros y más velocidad

Justificación. (i) El corpus está en inglés, por lo que un modelo *uncased* en inglés cubre el idioma y su vocabulario WordPiece; (ii) las noticias son cortas (percentil 95 = 71 sub-palabras), de modo que no se necesita un contexto largo; (iii) la restricción de cómputo (entrenamiento en una GPU de consumo, en este caso la GPU integrada de un Mac con Apple MPS) favorece un modelo compacto; (iv) DistilBERT se obtuvo por destilación de conocimiento a partir de BERT (Sanh et al., 2019), por lo que hereda gran parte de su capacidad.

4.3 Arquitectura del modelo

Componente	Detalle
Embeddings	Embeddings de tokens (vocabulario ≈ 30 k) + embeddings posicionales, dimensión 768
Encoder	**6 capas Transformer**; en cada una, auto-atención multi-cabeza con **12 attention heads** (64 dimensiones por cabeza) y red feed-forward de 3.072 unidades
Auto-atención	Cada token genera Query, Key y Value; los pesos de atención son $\mathrm{softmax}(QK^T/\sqrt{d_k})$ y mezclan los Values. Las proyecciones se llaman `q_lin`, `k_lin`, `v_lin`, `out_lin` en DistilBERT
Cabeza de clasificación	`pre_classifier` (768→768) + ReLU + dropout + `classifier` (768→4); se usa la representación del token `[CLS]`
Salida	4 logits → softmax → clase (entropía cruzada como pérdida)

4.4 Configuración de LoRA (PEFT)

LoRA (*Low-Rank Adaptation*, Hu et al., 2021) **congela** los pesos preentrenados W_0 y aprende una actualización de bajo rango $\Delta W = (\alpha/r)\cdot B\cdot A$, donde A ($r \times d$) y B ($d \times r$) son las únicas matrices entrenables; B se inicializa en cero, de modo que al comenzar el entrenamiento el modelo es idéntico al original. El *backpropagation* sigue calculando gradientes a través de todo el grafo, pero solo se actualizan los adaptadores y la cabeza.

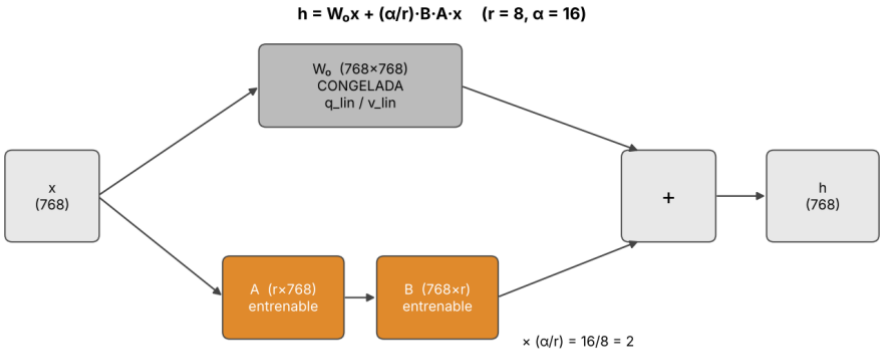


Figura 8. Esquema de un adaptador LoRA aplicado a una proyección de atención (q_lin o v_lin).

Tabla 9. Hiperparámetros de LoRA y su argumento

Hiperparámetro	Valor	Argumento
Rango `r`	8	Tarea simple (4 clases temáticas, textos cortos): un rango bajo captura la adaptación necesaria con pocos parámetros; rangos mayores aumentarían parámetros sin evidencia de beneficio
`lora_alpha`	16	Factor de escala α/r = 2; regla habitual α = 2r
`target_modules`	`q_lin`, `v_lin`	Proyecciones *query* y *value* de la auto-atención en las 6 capas (12 adaptadores); es la elección estándar del paper original
`modules_to_save`	`pre_classifier`, `classifier`	La cabeza de clasificación es nueva (inicialización aleatoria) y se entrena completa
`lora_dropout`	0,1	Regularización de los adaptadores
`task_type`	`SEQ_CLS`	Clasificación de secuencias
`bias`	`none`	No se entrenan sesgos

Análisis de parámetros

Bloque	Cálculo	Parámetros
Adaptadores LoRA (12 módulos: q_lin y v_lin × 6 capas)	12 × (768×8 + 8×768) = 12 × 12.288	147.456
`pre_classifier`	768×768 + 768	590.592
`classifier`	768×4 + 4	3.076
Total entrenable		**741.124**
Modelo original (con cabeza de 4 clases)		66.956.548
Total tras PEFT (incluye copias de los módulos guardados)		67.697.672
% entrenable	741.124 / 67.697.672	**1,09%** (1,11% respecto del modelo original)

El porcentaje (≈ 1,1%) cumple holgadamente el criterio de la consigna (< 3%). Es importante notar que **la mayor parte de los parámetros entrenables (80%) pertenece a la cabeza `pre_classifier`, no a los adaptadores LoRA** (147.456 parámetros, ≈ 20%). Los adaptadores en sí son ≈ 0,22% del modelo.

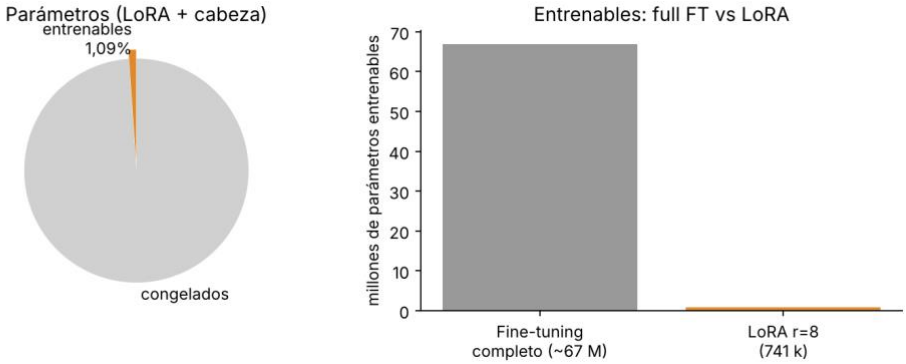


Figura 9. Proporción de parámetros entrenables y comparación con un fine-tuning completo.

4.5 Ciclo de entrenamiento

Elemento	Valor
Datos	Train: 7.200 (90%) · Validación: 800 (10%, estratificado) · Test: 2.000 (intocable)
Optimizador / planificador	AdamW (`weight_decay=0.01`) con decaimiento lineal del learning rate (valor por defecto de Trainer)
Learning rate	0.0002 (mayor que el 2e-5 típico del fine-tuning completo, porque solo se entrenan adaptadores y cabeza)
Épocas / batch / pasos	3 épocas · batch 32 · 225 pasos por época (675 en total)
Selección del checkpoint	`load_best_model_at_end=True`, métrica `f1_macro` en validación por época
Precisión	fp32: `fp16` se activa solo con CUDA, y el entrenamiento corrió en Apple MPS
Hardware / software	Apple MPS (GPU del Mac) · PyTorch 2.15.0 · PEFT 0.21.2
Semilla	42 (`set_seed`)
Tiempo	**219.4 s (3.7 min)** de entrenamiento; 8.7 s de inferencia sobre el test

Tabla 10. Evolución en validación por época

Época	Val loss	Val F1 macro
1	0.2668	0.9099
2	0.2577	0.9161
3	0.2578	0.9170

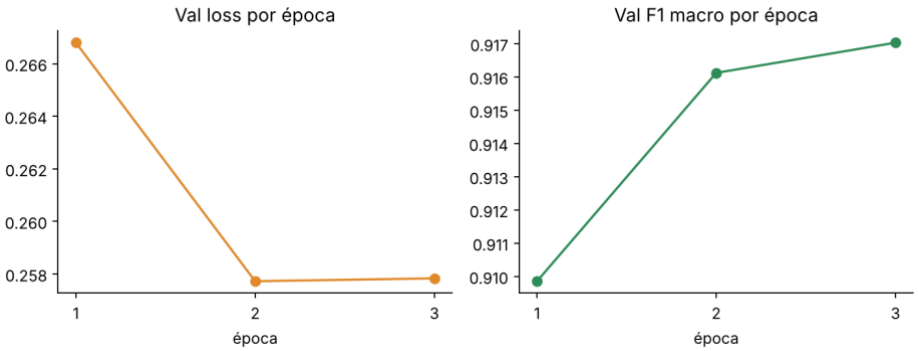


Figura 10. Pérdida y F1 macro en validación por época.

Interpretación. La pérdida de validación baja de 0.2668 a 0.2577 en la segunda época y se **estabiliza** en la tercera (0.2578), mientras el F1 macro sigue creciendo levemente (0.9099 → 0.9170). La pérdida final de entrenamiento (0.2724, último valor registrado, que es un promedio con *dropout* activo) es **cercana a la de validación (0.2578)**:

no hay señal de sobreajuste, a diferencia del MLP del Módulo 1. La brecha entre el F1 de validación (0,9170, solo 800 documentos) y el de test (0,9082) se encuentra dentro del ruido muestral esperable.

> **Limitación de la evidencia.** El proyecto guardó solo la última pérdida de entrenamiento y las métricas de validación por época, no la curva de pérdida por paso. Para futuras corridas se recomienda registrar `trainer.state.log_history` completo y graficar ambas curvas.

4.6 Artefacto resultante

El adaptador se guarda con `model.save_pretrained("lora_adapter")`: contiene únicamente los pesos de LoRA y de la cabeza (≈ 741 mil parámetros, del orden de unos pocos MB) más `adapter_config.json`, y se carga sobre `distilbert-base-uncased` para inferencia. Su configuración (PEFT 0.21.2) confirma: `peft_type: LORA`, `r: 8`, `lora_alpha: 16`, `lora_dropout: 0.1`, `target_modules: [q_lin, v_lin]`, `task_type: SEQ_CLS`, `base_model_name_or_path: distilbert-base-uncased` (Anexo E).

5. Evaluación comparativa

5.1 Protocolo de comparación (manzanas con manzanas)

- Ambos modelos predicen sobre el **mismo `ag_news_test.csv`** (2.000 noticias, 500 por clase) y con el mismo orden de clases; las predicciones se guardaron por noticia, lo que permite análisis pareados.
- El test se usó **una sola vez** por modelo, después de elegir hiperparámetros en validación interna.
- **Verificación de reproducibilidad del baseline:** al reentrenar TF-IDF (1,2) + MultinomialNB con el corpus preprocesado y los mismos parámetros se obtienen **25.400 características y predicciones idénticas al 100%** a las guardadas (accuracy 0,8960).
- Las matrices de confusión, el análisis por clase y las pruebas estadísticas usan las predicciones guardadas de ambos modelos.

5.2 Tabla comparativa de métricas

Tabla 11. Métricas globales sobre el mismo test

Métrica	A: TF-IDF + MultinomialNB	B: DistilBERT + LoRA	Δ (B – A)
Precision (weighted)	0.8964	0.9091	+0.0127
Recall (weighted)	0.8960	0.9080	+0.0120
F1 (weighted) ★	0.8960	0.9082	+0.0122
Precision (macro)	0.8964	0.9091	+0.0127
Recall (macro)	0.8960	0.9080	+0.0120
F1 (macro)	0.8960	0.9082	+0.0122
Accuracy	0.8960	0.9080	+0.0120

★ *Métrica principal. Macro y weighted coinciden por el balance perfecto de clases.*

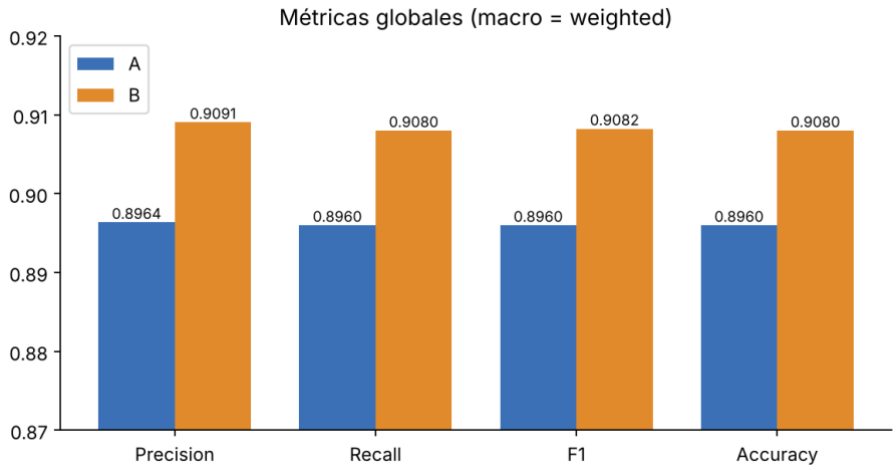


Figura 11. Métricas globales de ambos modelos.

5.3 Desempeño por clase

Tabla 12. Precision, Recall y F1 por clase (A = TF-IDF+NB, B = DistilBERT+LoRA)

Clase	P (A)	R (A)	F1 (A)	P (B)	R (B)	F1 (B)	ΔF1
Business	0.8395	0.8680	0.8535	0.8317	0.8700	0.8504	-0.0031
Sci_Tech	0.8803	0.8680	0.8741	0.8931	0.8860	0.8896	+0.0154

Clase	P (A)	R (A)	F1 (A)	P (B)	R (B)	F1 (B)	ΔF1
Sports	0.9490	0.9680	0.9584	0.9704	0.9840	0.9772	+0.0187
World	0.9167	0.8800	0.8980	0.9409	0.8920	0.9158	+0.0179

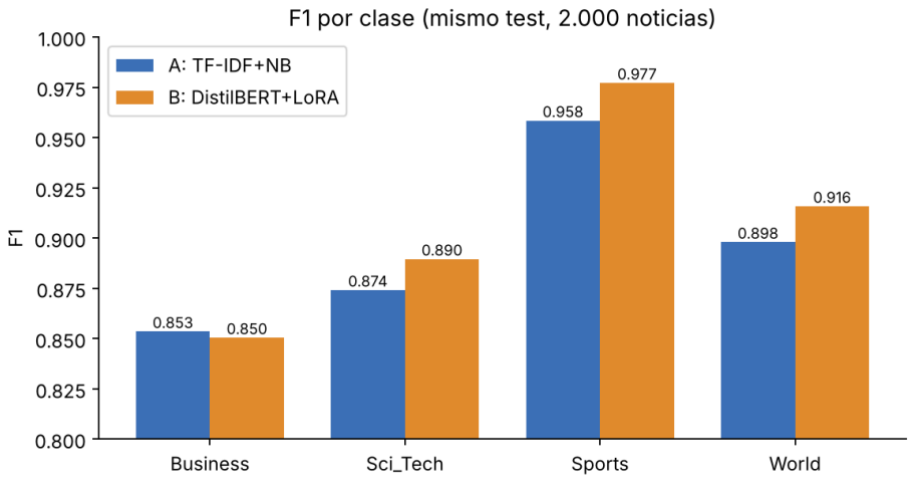


Figura 12. F1 por clase.

El Transformer **mejora tres de las cuatro clases**: World (+0,0178), Sports (+0,0188) y Sci_Tech (+0,0155). **En Business no mejora** (F1 de 0,8504 frente a 0,8535, una diferencia dentro del ruido). Es decir, la ganancia global proviene de resolver mejor las fronteras de Sports y World, no de la frontera Business ↔ Sci_Tech, que sigue siendo el cuello de botella de ambos modelos.

5.4 Matrices de confusión comparadas

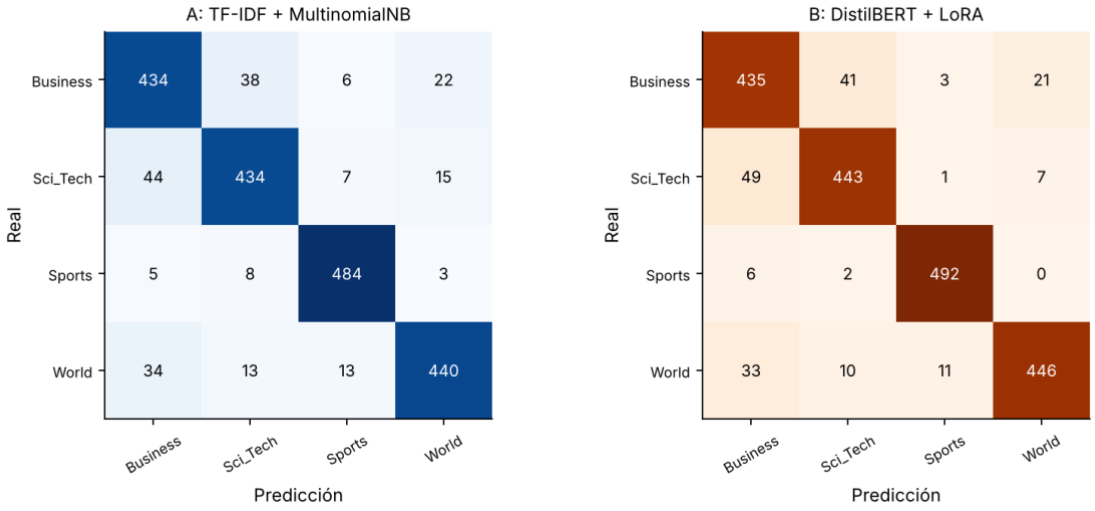


Figura 13. Matrices de confusión lado a lado (test).

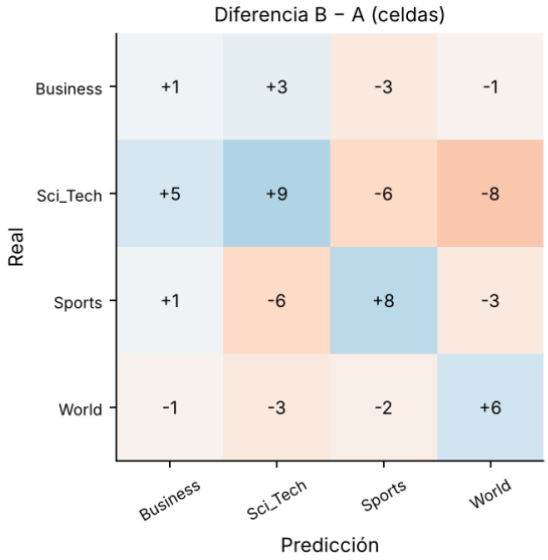


Figura 14. Diferencia celda a celda (B − A): azul = más casos en B, rojo = menos. En la diagonal, más casos es mejor; fuera de ella, menos es mejor.

Lectura conjunta. El Transformer reduce los errores que involucran Sports y World (p. ej. Sci_Tech → World baja de 15 a 7 y Sci_Tech → Sports de 7 a 1, y suben los aciertos en Sports a 492 sobre 500). Pero la confusión principal **permanece e incluso crece levemente**: Sci_Tech → Business pasa de 44 a 49 y Business → Sci_Tech de 38 a 41. Esto es consistente con que muchas noticias de empresas tecnológicas (fusiones, resultados, productos) son **intrínsecamente ambiguas** entre ambas etiquetas, y con la presencia de ruido de etiquetado en AG News (ver 5.7).

5.5 ¿La diferencia es estadísticamente significativa?

Con un solo test de 2.000 noticias y una semilla, conviene cuantificar la incertidumbre. Como ambos modelos predicen las mismas noticias, se usan métodos **pareados**:

Tabla 13. Acuerdo entre modelos sobre las 2.000 noticias

	B acierta	B falla
A acierta	1743 (ambos aciertan)	49 (solo A)
A falla	73 (solo B)	135 (ambos fallan)

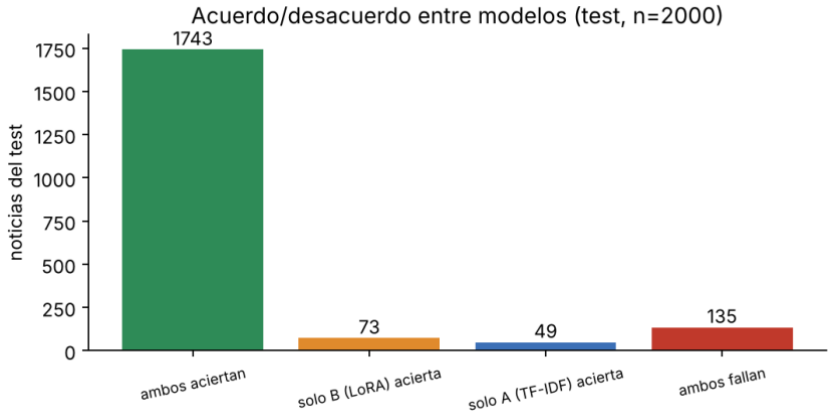


Figura 15. Acuerdo y desacuerdo entre los dos modelos.

- **Prueba de McNemar** (con corrección de continuidad): $\chi^2 = 4.34$, **p = 0.037** (73 aciertos exclusivos de B frente a 49 de A). Se rechaza la igualdad al 5%, pero no al 1%.

- **Bootstrap pareado** (5.000 remuestreos del test): diferencia de F1 macro **+0.0122**, IC 95% **[+0.0015, +0.0232]**. El intervalo excluye el cero, pero apenas: el límite inferior es de solo 0,15 puntos.
- IC 95% individuales de F1 macro (bootstrap): A [0.8819, 0.9092]; B [0.8955, 0.9203]. Se solapan ampliamente, por lo que la evaluación de cada modelo por separado no basta para afirmar la superioridad; el análisis pareado sí.

> **Conclusión estadística.** B supera a A de forma **detectable pero modesta y de magnitud incierta** (entre 0,2 y 2,3 puntos de F1 con 95% de confianza). Con una sola semilla de entrenamiento no se midió la variabilidad entre corridas del Transformer, por lo que la significancia real podría ser menor (sección 6.3).

5.6 Complejidad léxica y cobertura del vocabulario

Para contrastar la hipótesis de que el Transformer gana por manejar mejor el vocabulario poco frecuente, se midió la **cobertura léxica** del test respecto del vocabulario de train (texto lematizado, la representación que ve el baseline):

Indicador	Valor
Tipos (lemas distintos) en train	17.719
Tipos distintos en test	9.098
Tokens del test fuera del vocabulario de train (OOV)	3.05%
Noticias del test con al menos un token OOV	56%

Tabla 14. Cobertura léxica y longitud según quién acierta

Grupo de aciertos	Noticias	% tokens OOV (prom.)	Longitud media (tokens)
ambos aciertan	1743	3.13%	37.2
solo B (LoRA) acierta	73	3.44%	34.5
solo A (TF-IDF) acierta	49	3.43%	35.4
ambos fallan	135	3.12%	37.2

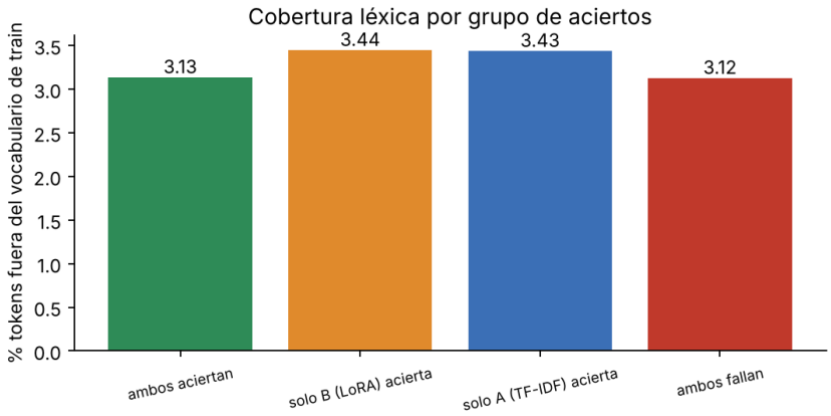


Figura 16. Tasa de tokens fuera de vocabulario por grupo de aciertos.

Hallazgo (honesto, y en parte contrario a la hipótesis inicial). Solo ≈ 3.1% de los tokens del test no aparece en el train, y la tasa de OOV es **prácticamente la misma** en las noticias que solo el Transformer acierta (3,44%) que en las que solo el baseline acierta (3,43%) o en las que ambos aciertan (3,13%). Es decir, **el vocabulario desconocido no explica la ventaja del Transformer en este corpus**. Con 8.000 noticias de un dominio estable y un vocabulario distintivo por tema, la cobertura léxica del baseline es suficiente. La cobertura del tokenizer WordPiece (que casi elimina el OOV por diseño) no se midió directamente porque el tokenizer no estaba disponible en el entorno de análisis; sería un ítem de validación futura.

Efecto de la longitud del documento

Cuartil de longitud (tokens)	Accuracy A	Accuracy B	Δ
10–31	86.3%	89.0%	+2.7 pp
32–37	90.1%	91.5%	+1.4 pp
38–42	92.1%	91.4%	-0.6 pp
43–137	90.1%	91.3%	+1.1 pp

La mayor mejora se concentra en el **cuartil de noticias más cortas (≤ 31 tokens)**: de 86,3% a 89,0% (+2,7 puntos porcentuales). Un texto corto aporta pocos términos a la bolsa de palabras, por lo que un error de una palabra ambigua pesa mucho; el Transformer compensa con el contexto de toda la secuencia. En el tercer cuartil el baseline incluso supera levemente al Transformer (92,1% vs 91,4%). Son tendencias exploratorias: cada cuartil tiene ≈ 500 noticias, y las diferencias no se testearon individualmente.

5.7 Semántica de secuencias e interpretabilidad: qué capturó B que A ignoró

La consigna pide explicar con ejemplos qué entendió el Transformer que el modelo clásico ignoró. Como no se guardaron los pesos de atención, el análisis es **por casos**: para el baseline se calculó la **contribución de cada término a la decisión errónea** (diferencia de log-probabilidad entre la clase predicha y la real, ponderada por el peso TF-IDF); para el Transformer no se inspeccionó la atención, por lo que su explicación es una **hipótesis a partir del texto**, no una verificación.

Tabla 15. Ejemplos del test acertados solo por el Transformer (A erró)

Texto (resumido)	Real	A (TF-IDF)	B (LoRA)	Términos que empujaron a A y lectura
"Olympics: Hansen still strong enough to take bronze. Every ounce of his energy was expended, leaving an empty fuel tank…"	Sports	Business	Sports	*energy, fuel, strong, bolster, grow*: léxico de empresas/energía usado en sentido metafórico. B parece usar el contexto ("Olympics", "bronze", natación).
"Barents Sea Under Threat from Fishing, Oil (Reuters)… overfishing, oil and gas exploration and Soviet-era radioactive waste, the U.N. Environment Program said…"	Sci_Tech	World	Sci_Tech	*oil, soviet, threat, radioactive*: A asocia "oil" y "soviet" con mundo/negocios; B reconoce el tema ambiental/científico.
"Atlantis 'Evidence' Found in Spain, Ireland. In 360 B.C. the Greek philosopher Plato described an island…"	Sci_Tech	Sports	Sci_Tech	*spain, greek, ireland, city, place*: nombres de países empujan a Sports (selecciones). B capta el sentido de "evidencia/hallazgo".
"Producer sues for Rings profits. Hollywood producer Saul Zaentz sues the producers of The Lord of the Rings for $20m in royalties."	World	Business	World	*sue, profit, producer, ring, royalty*: léxico económico-legal. **Etiqueta discutible** (podría ser Business); B coincide con la etiqueta.
"Dropping Hyphen, Some Great Old Stores Become Just Macy #39;s…"	Business	Sci_Tech	Business	*discover, old, school, great*: palabras genéricas sin señal de negocio. Nota: B recibe el texto con "#39;" ya reparado.
"EU, US Talks on Aircraft Aid Grounded. US and EU negotiators disagreed… state aid for aircraft rivals Airbus and Boeing…"	Business	World	Business	*aid, election, presidential, gather*: A se deja llevar por "presidential election" (mundo/política).

Patrón común. Los errores del baseline suelen deberse a **una o dos palabras con fuerte sesgo hacia otra clase** (*oil*, *profit*, *fuel*, nombres de países) que, fuera de contexto, apuntan a la clase equivocada. El Transformer, al ponderar cada token en función de los demás mediante auto-atención y *embeddings* contextuales, tiende a resolver la polisemia ("fuel tank" en un artículo de natación) y a dar peso a la señal temática global ("Olympics", "bronze"). Esto es compatible con la hipótesis de **semántica de secuencias**, aunque confirmarla requeriría inspeccionar la atención (p. ej. *attention rollout* o SHAP).

Tabla 16. Ejemplos acertados solo por el baseline (B erró)

Texto (resumido)	Real	A	B	Lectura
"GM Plans to Use Woods More Creatively (AP)… General Motors Corp. is thrilled that Tiger Woods will promote Buick…"	Sports	Sports	Business	B se deja arrastrar por "General Motors", "Buick", "automaker": el léxico corporativo domina.
"Clouds darken PeopleSoft conference… rival Oracle Corp.'s $7.7 billion…"	Business	Business	Sci_Tech	B prioriza el contexto tecnológico (software empresarial); la etiqueta es Business.
"Times to go completely compact. The Times newspaper has announced that it is to move on from … broadsheet…"	Sci_Tech	Sci_Tech	Business	Etiqueta Sci_Tech muy discutible; A acierta quizás por azar léxico.

El Transformer **no es estrictamente superior**: en 49 noticias el baseline acierta y B falla. Allí el contexto "corporativo" del texto puede pesar más que el tema real (un golfista promocionando una marca), un tipo de error que la bolsa de palabras evita porque aísla las palabras clave del tema (*golf*, *Woods*).

Tabla 17. Ejemplos en los que ambos modelos fallan (posible ruido de etiquetado)

Texto (resumido)	Real	A	B
"BSkyB sees profits rise after strong subscriber growth (AFP)…"	World	Business	Business
"Maddux Wins No. 302, Baker Wins No. 1,000. Greg Maddux pitched the Chicago Cubs into the lead…"	World	Sports	Sports
"E.U. Regulators Approve Oracle's PeopleSoft Bid…"	Sci_Tech	Business	Business
"Symantec Said to Be in Talks for Veritas…"	Business	Sci_Tech	Sci_Tech

En **135 noticias fallan ambos modelos**. Una lectura manual de varios de estos casos sugiere que muchos errores compartidos **no son errores del modelo sino ruido de etiquetado de AG News**: una nota de béisbol etiquetada como World, o una de resultados corporativos de una emisora británica etiquetada como World, son clasificadas coherentemente por ambos modelos como Sports y Business. Esto establece un **techo práctico de desempeño por debajo del 100%** y explica por qué incluso un Transformer queda alrededor del 91%.

5.8 Análisis de eficiencia: tiempo de entrenamiento frente a desempeño

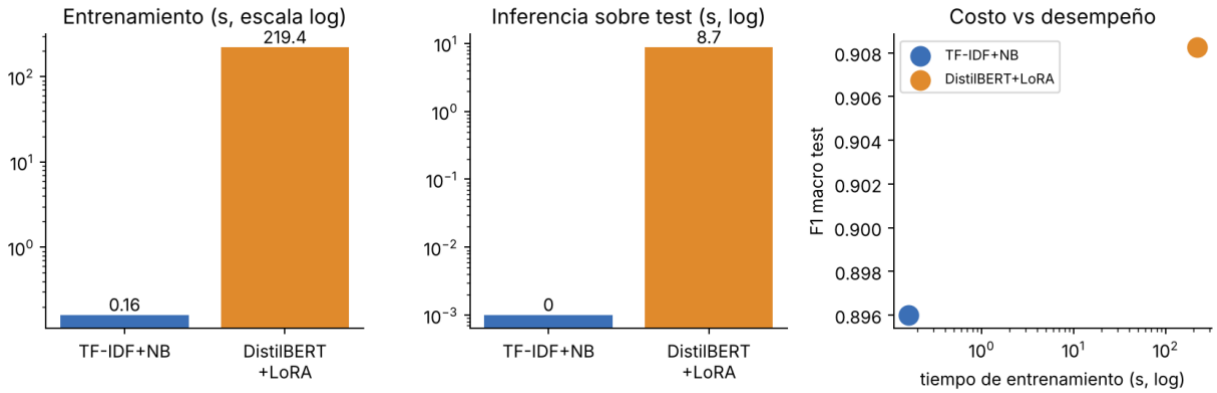


Figura 17. Costo computacional y desempeño de ambos modelos (escalas logarítmicas en tiempo).

Dimensión	A: TF-IDF + NB	B: DistilBERT + LoRA	Relación
Tiempo de entrenamiento	0.16 s	219.4 s (3.7 min)	≈ 1.371×
Inferencia sobre 2.000 noticias	< 0,01 s	8.7 s	> 800×
Hardware	CPU	GPU (Apple MPS)	—
Parámetros entrenados	25.400 probabilidades por clase	741.124 (1,09%)	—
Artefacto a desplegar	Vectorizador + NB (livianos)	Modelo base + adaptador (≈ MB)	—

Dimensión	A: TF-IDF + NB	B: DistilBERT + LoRA	Relación
F1 weighted	0.8960	0.9082	+0.0122 (+1.22 pp)
Interpretabilidad	Alta (términos por clase)	Baja (requiere técnicas posteriores)	—

Costo marginal: **≈ 179 segundos de entrenamiento por cada punto porcentual de F1 ganado** (219,4 s / 1,22 pp), y una inferencia unas 800 veces más lenta. Dos aclaraciones para leer estas cifras con rigor: (i) el denominador del baseline (0,16 s) es tan pequeño que el cociente de 1.371× es muy sensible a la medición; lo robusto es el orden de magnitud (milisegundos-segundos vs minutos); (ii) los tiempos miden solo el ajuste del modelo: **no incluyen la lematización con spaCy del baseline ni la tokenización del Transformer**. En términos absolutos, además, ≈ 3,7 minutos de entrenamiento en una GPU integrada de consumo es un costo bajo: LoRA permite que el costo del Transformer sea razonable en un entorno de recursos limitados.

5.9 Respuesta a la pregunta central: el porqué desde los tres ejes

Eje	Evidencia	Veredicto
Complejidad léxica	Solo 3.1% de OOV en test; la tasa es igual en aciertos exclusivos de A, de B y comunes; 6.968 hápax en train (39% del vocabulario) que `min_df=2` descarta sin dañar al baseline	**No explica** la diferencia: el vocabulario del test está bien cubierto por el train
Cobertura del vocabulario	TF-IDF usa 25.400 términos aprendidos y descarta stop-words y hápax; WordPiece usa un vocabulario fijo que no deja palabras fuera (no medido directamente)	**Efecto limitado y no demostrado** en este corpus
Semántica de las secuencias	Los errores corregidos por B son polisemias y usos metafóricos (fuel, oil, nombres de países); la mejora se concentra en noticias cortas y en las fronteras de Sports y World	**Es la explicación más consistente**, aunque sin verificación por atención
Techo por ruido de etiquetas	135 noticias falladas por ambos; varias con etiqueta discutible; Business ↔ Sci_Tech no mejora	**Limita a ambos modelos**

En síntesis: el Transformer gana poco porque en AG News el tema está en gran parte determinado por el vocabulario, que la bolsa de palabras ya captura bien; su ventaja aparece donde la palabra clave engaña (polisemia, metáforas, textos cortos). Ninguno de los dos modelos supera el ruido inherente de las etiquetas.

6. Conclusiones, limitaciones y mejoras futuras

6.1 Hallazgos principales

1. **El pipeline de datos importa:** el 33% de los documentos de train traía ruido de codificación y el 21% firmas de agencia. Limpiar antes de modelar evitó que n-gramas espurios (*amp*, *39*, *ap ap*) dominaran el vocabulario.
2. **Baseline sólido:** TF-IDF (1,2)-gramas + Naive Bayes alcanza F1 weighted **0.8960** con 0.16 s de entrenamiento en CPU y es totalmente interpretable.
3. **Transformer + LoRA:** DistilBERT adaptado entrenando solo **1.09%** de los parámetros alcanza **0.9082** (+0.0122), en 3.7 minutos.
4. **La mejora es real pero pequeña:** IC 95% pareado [+0.0015, +0.0232]; McNemar p = 0.037. El baseline sigue ganando en 49 noticias.
5. **Por qué:** la ventaja de B no se debe a cobertura léxica sino a semántica contextual (polisemia, metáforas, textos cortos); la frontera Business ↔ Sci_Tech y el ruido de etiquetas limitan a ambos.
6. **Sin sobreajuste en M4**, a diferencia del MLP del M1, gracias a pocos parámetros entrenables, validación por época, selección del mejor checkpoint y solo 3 épocas.

6.2 Recomendación al negocio

La pregunta del curso es: *¿qué le recomendás al negocio y con qué argumento?* La respuesta depende del contexto de decisión:

Escenario	Recomendación	Argumento
Enrutamiento editorial a gran escala, bajo costo, sin GPU, necesidad de auditar decisiones	**Baseline TF-IDF + NB (o SVM)**	Entrena en décimas de segundo en CPU, inferencia casi instantánea, 89,6% de F1 y explicable por términos
Cada punto de F1 tiene valor alto (p. ej. etiquetado que alimenta recomendaciones o publicidad) y hay GPU disponible	**DistilBERT + LoRA**	+1,2 puntos con 1,1% de parámetros entrenables y un adaptador liviano; costo de ≈ 4 minutos de entrenamiento
Se quiere lo mejor de ambos	**Híbrido por confianza**	Usar el baseline como filtro rápido y derivar al Transformer solo las noticias de baja confianza (p. ej. Business ↔ Sci_Tech). *Propuesta no evaluada en este proyecto*

Dado el tamaño de la mejora, la **recomendación por defecto es mantener el baseline** y adoptar el Transformer solo si el negocio valora ese punto de F1 por encima del costo de operar GPU, monitoreo y menor interpretabilidad.

6.3 Limitaciones del sistema y de la evidencia

- **Una sola semilla y una sola corrida de LoRA:** no se midió la variabilidad entre corridas del Transformer; la mejora de +1,2 puntos podría variar de una semilla a otra.
- **Un solo corpus** (AG News recortado, 8.000/2.000), en inglés y con textos cortos. No se puede extrapolar a textos largos, otros idiomas ni a dominios con desbalance de clases (donde macro y weighted diferirían y el Accuracy sería engañoso).
- **Sin búsqueda de hiperparámetros para LoRA:** se usaron valores estándar (r=8, α=16, lr=2e-4, 3 épocas) sin explorar r, módulos objetivo ni lr. El baseline sí tuvo una grilla de 18 combinaciones, lo que podría favorecerlo levemente en la comparación.

- **Validaciones pequeñas:** 1.600 documentos (M3) y 800 (M4); el error estándar de ≈ 0,75–1,0 puntos impide discriminar configuraciones cercanas.
- **Interpretabilidad del Transformer solo por casos:** no se inspeccionaron pesos de atención ni se calculó SHAP.
- **Mediciones de tiempo parciales:** excluyen preprocesamiento y tokenización; un único hardware (Apple MPS) y una sola medición; el denominador del baseline es muy pequeño.
- **Ruido de etiquetado** en AG News: fija un techo de desempeño y puede sesgar la lectura de las comparaciones.
- **Curva de entrenamiento incompleta** en M4: solo se guardó la última pérdida de entrenamiento.

6.4 Líneas de mejora futuras

- Repetir el entrenamiento con **5 o más semillas** y reportar media ± desvío; extender McNemar/bootstrap a las repeticiones.
- Explorar **r ∈ {4, 8, 16, 32}**, otros módulos objetivo (`k_lin`, `out_lin`, capas FFN), learning rate y número de épocas con validación cruzada.
- Comparar con **BERT-base / RoBERTa-base** y con *full fine-tuning* de DistilBERT para cuantificar qué se pierde al usar LoRA.
- **Interpretabilidad real:** *attention rollout*, SHAP o LIME sobre los casos de la Tabla 15; medir la tasa de fragmentación de WordPiece.
- **Auditoría de etiquetas** de las 135 noticias en las que ambos modelos fallan (aprendizaje con ruido de etiquetas, *confident learning*).
- Sistema **híbrido por confianza** (baseline + Transformer) y evaluación en un escenario desbalanceado con F1 macro/weighted y curvas precisión-recall.
- Registrar costos completos (preprocesamiento, tokenización, memoria máxima, energía) y probar inferencia por lotes en CPU.

6.5 Referencias

1. Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. *Advances in Neural Information Processing Systems 28 (NeurIPS)*.
2. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 30*.
3. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*.
4. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*.
5. Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*.
6. Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization (AdamW). *ICLR*.
7. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
8. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.
9. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 32*.
10. Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. *EMNLP: System Demonstrations*.

11. Mangrulkar, S., et al. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. Hugging Face.
12. Honnibal, M., & Montani, I. spaCy: Industrial-strength Natural Language Processing in Python. Explosion AI.
13. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523.
14. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157.
15. Efron, B. (1979). Bootstrap methods: another look at the jackknife. *The Annals of Statistics, 7*(1), 1–26.

Anexo A. Pre-entrega 1 (Módulo 1): pipeline de entrenamiento, validación y clasificador base

Objetivo de la entrega: construir la infraestructura técnica inicial del proyecto, asegurando un ciclo de entrenamiento funcional en PyTorch, con validación separada y registro de métricas por época. El foco no es la complejidad del modelo sino la **calidad del pipeline**.

A.1 Entorno y reproducibilidad

Elemento	Valor
Versión de PyTorch	2.15.0
Dispositivo detectado	`mps` (selección automática en orden cuda → mps → cpu; en este equipo, la GPU de Apple)
Semilla	42 (`torch.manual_seed`, `np.random.seed`)
Learning rate	0.001 (Adam)
Épocas / batch	15 épocas · batch 64
Pérdida / métrica	`CrossEntropyLoss` (incluye softmax) · Accuracy

A.2 Datos y arquitectura

Se utilizó AG News (solo el train; el test **no se toca** en M1). El train se divide en entrenamiento y validación estratificados (6.400 / 1.600). El texto se vectoriza con TF-IDF (`max_features=5000`, `stop_words='english'`, `sublinear_tf=True`) **ajustado solo con la parte de entrenamiento**, y la matriz se densifica a tensores `float32` para alimentar la red.

Capa	Detalle	Parámetros
Entrada	5.000 características TF-IDF	—
`Linear(5000, 256)` + ReLU + `Dropout(0.3)`	Capa oculta con activación no lineal ReLU (evita el problema de XOR y el *vanishing gradient* de Sigmoid)	1.280.256
`Linear(256, 4)`	Capa de salida: 4 logits (uno por clase)	1.028
Total		**1.281.284**

A.3 Ciclo de entrenamiento y validación

Se implementó explícitamente cada fase, con los errores comunes del módulo controlados (limpieza de gradientes, consistencia de dispositivos con `.to(device)`):

```
def train_epoch():
    model.train()
    for xb, yb in dl_tr:
        xb, yb = xb.to(device), yb.to(device)   # consistencia de dispositivo
        optimizer.zero_grad()                   # limpiar gradientes acumulados
        out = model(xb)                         # forward
        loss = criterion(out, yb)               # pérdida (entropía cruzada)
        loss.backward()                         # backward: autograd / regla de la cadena
        optimizer.step()                        # actualización con Adam

@torch.no_grad()
def evaluate(dl):                               # validación sin gradientes, model.eval()
    ...                                         # devuelve (loss, accuracy) en datos no vistos
```

A.4 Resultados por época

Tabla A1. Métricas de entrenamiento y validación del MLP (15 épocas)

Época	Train loss	Train acc	Val loss	Val acc
1	1.0499	0.7641	0.5491	0.8850
2	0.3520	0.9155	0.3347	0.8988
3	0.1999	0.9472	0.3110	0.8988
4	0.1281	0.9686	0.3149	0.8962
5	0.0840	0.9819	0.3267	0.8931
6	0.0559	0.9902	0.3390	0.8906
7	0.0382	0.9959	0.3582	0.8869
8	0.0271	0.9975	0.3734	0.8862
9	0.0197	0.9983	0.3895	0.8838
10	0.0149	0.9986	0.4036	0.8831
11	0.0116	0.9991	0.4172	0.8812
12	0.0095	0.9998	0.4300	0.8838
13	0.0075	0.9997	0.4425	0.8825
14	0.0063	0.9995	0.4546	0.8794
15	0.0052	0.9998	0.4668	0.8800

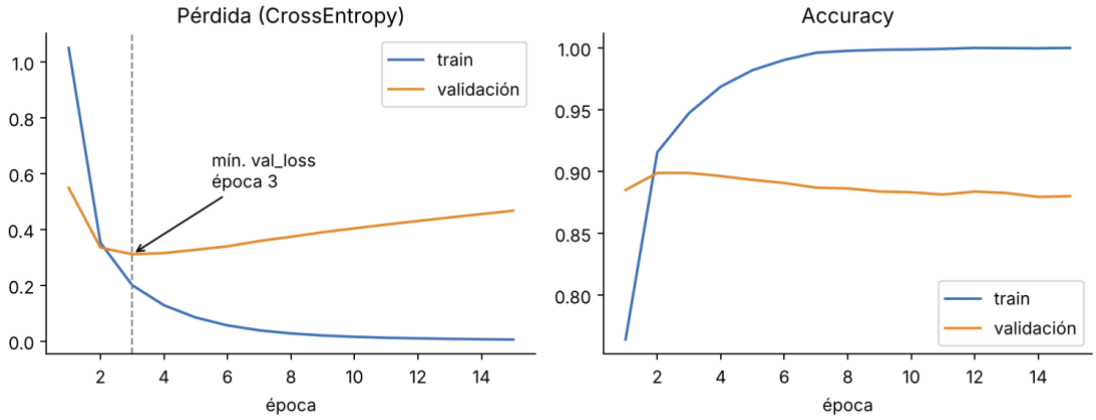


Figura A1. Curvas de pérdida y accuracy del MLP (entrenamiento vs. validación).

A.5 Interpretación de la curva de pérdida

- **¿Bajó la pérdida durante las épocas?** Sí: la pérdida de entrenamiento baja de 1.050 a 0.0052 y el accuracy de entrenamiento llega a 99.98%. El ciclo de entrenamiento funciona y el modelo aprende.
- **Pero hay sobreajuste.** La pérdida de validación alcanza su mínimo en la **época 3** (0.3110; accuracy de validación máxima 0,8988 en las épocas 2 y 3) y luego **sube de forma sostenida** hasta 0.4668, mientras la de entrenamiento sigue bajando. El accuracy de validación cae a 88.0%. Con 1,28 M de parámetros y solo 6.400 documentos, la red memoriza el train.
- **Qué se aprendió para el resto del proyecto:** conviene *early stopping* (≈ época 3), regularización (dropout, *weight decay*) y monitorear siempre la validación. Esa lección se aplicó en M4 (validación por época, mejor checkpoint, 3 épocas).

> **Precaución.** El accuracy de validación de M1 (≈ 0,899, sobre 1.600 documentos del train) **no es comparable** con las métricas de test de M3/M4 (sobre 2.000 documentos distintos): son conjuntos diferentes. M1 es un clasificador base para validar el pipeline, no un competidor del informe final.

A.6 README de la pre-entrega 1 (consolidado)

Requisito de la consigna	Contenido
Versión de PyTorch	2.15.0
Learning rate elegido	1e-3 (Adam)
Interpretación de la curva	La pérdida de entrenamiento desciende de forma monótona; la de validación toca fondo en la época 3 y luego sube: sobreajuste. Se recomienda detener el entrenamiento alrededor de esa época.
Estructura del repositorio	`data/`, `outputs/` (JSON, figuras, PDF), script principal, `README.md`, `requirements.txt`
Registro	`outputs/m1_history.json` (loss y accuracy por época) y `outputs/figs/m1_curvas.png`

Anexo B. Matriz de cumplimiento de rúbricas

Se contrasta cada criterio de evaluación de las cinco entregas con la evidencia de este documento (aprobación: 70/100 en cada una).

B1. Pre-entrega 1: pipeline de entrenamiento y clasificador base

Criterio (peso)	Evidencia
Estructura del repositorio y profesionalidad (20%)	Anexo C: estructura, `README.md`, `requirements.txt`
Configuración de PyTorch y dispositivo (15%)	A.1: selección automática cuda/mps/cpu, semilla, versión
Arquitectura base `nn.Module` (15%)	A.2: MLP 5000→256→4 con ReLU y dropout
Pipeline con autograd y Adam (35%)	A.3: forward, loss, `zero_grad`, `backward`, `step`
Validación: pérdida y Accuracy (10%)	A.3–A.4: `evaluate` separada; tabla por época
README: versión, learning rate, curva (5%)	A.6

B2. Pre-entrega 2: corpus preprocesado y EDA

Criterio (peso)	Evidencia
Pipeline de preprocesamiento (35%)	2.1–2.2: `preprocess_text` (Regex + minúsculas + lematización spaCy), aplicada igual a train y test
EDA técnico: longitud, p95, clases (25%)	2.3: histograma, p95 = 52, balance perfecto
Calidad de n-gramas y ruido residual (20%)	Tabla 4: 20 bi-gramas y 20 tri-gramas sin artefactos; ruido residual = 0
Palabras frecuentes y stop-words (10%)	Tabla 5: 37 de 50 son stop-words
Informe PDF y comunicación (10%)	`EDA_NLP_GomezMariaLorena.pdf` y secciones 2.1–2.3

B3. Pre-entrega 3: clasificador supervisado con TF-IDF

Criterio (peso)	Evidencia
Pipeline completo y funcional (32%)	Sección 3 y notebook `M3_clasificador_tfidf.ipynb` ejecutado
TF-IDF e hiperparámetros (18%)	3.2–3.3: grilla de `max_features` × `ngram_range` × 3 clasificadores
Rigor estadístico, sin fuga (18%)	1.5, 3.2 y 5.1: `fit` solo en train; verificación de reproducibilidad
Calidad del modelo base y evaluación (18%)	3.4: `classification_report` y matriz de confusión
Documentación técnica (9%)	3.7, 3.6 y Anexo C (justificación, categorías difíciles, versiones)
Calidad del código y reproducibilidad (5%)	Anexo C: comandos y versiones

B4. Pre-entrega 4: fine-tuning con LoRA

Criterio (peso)	Evidencia
Selección del modelo base (20%)	4.2
Configuración de LoRA y módulos objetivo (25%)	4.4: r=8, α=16, `q_lin`/`v_lin`, `modules_to_save`
Evidencia de entrenamiento y estabilidad (15%)	4.5: pérdidas final de train/val, tabla por época
Análisis de parámetros y eficiencia PEFT (15%)	4.4: 741.124 entrenables (1,09%)
Benchmark final con baseline y eficiencia (15%)	5.2 y 5.8
Conclusión técnica y coherencia (10%)	5.8–5.9 y 6.2

B5. Proyecto final (capstone)

Criterio (peso)	Evidencia
Definición del problema y coherencia narrativa (20%)	Sección 1: objetivo, dataset, hipótesis, hilo conductor
Pipeline de preprocesamiento (18%)	Sección 2: limpieza, normalización, tokenización scikit-learn vs HF, reproducibilidad

Criterio (peso)	Evidencia
Baseline TF-IDF documentado (15%)	Sección 3
Solución avanzada con LoRA (22%)	Sección 4: modelo, atención, rango, backpropagation, ciclo PyTorch
Evaluación comparativa y eficiencia (20%)	Sección 5: tablas, matrices, mismo test, tiempo vs desempeño
Conclusiones, limitaciones y referencias (5%)	Sección 6

Anexo C. Guía de reproducción, estructura del repositorio y versiones

C.1 Estructura del repositorio

```
proyecto/
├── data/
│   ├── ag_news_train.csv            # 8.000 noticias
│   └── ag_news_test.csv             # 2.000 noticias (nunca se entrena con ellas)
├── proy_datasc_tp_mlg.py            # script único: etapas m1, m2, m3, m4, m4-pdf, m5
├── cargar_dataset.py                # loader de la cátedra
├── M3_clasificador_tfidf.ipynb      # notebook de la pre-entrega 3
├── requirements.txt
├── README.md
└── outputs/                         # se genera al correr
    ├── m1_history.json  m2_stats.json  m3_metrics.json  m4_metrics.json  m3_grid.csv
    ├── corpus_preprocesado_spacy.csv
    ├── lora_adapter/                # adapter_config.json + pesos del adaptador
    ├── figs/                        # figuras PNG
    └── *.pdf                        # EDA_NLP_*, *_Checkpoint4_NLP3, NLP_Capstone_*
```

C.2 Ejecución

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm

python proy_datasc_tp_mlg.py m1                  # Pre-entrega 1: pipeline PyTorch
python proy_datasc_tp_mlg.py m2 --apellido Gomez --nombre MariaLorena
python proy_datasc_tp_mlg.py m3                  # baseline TF-IDF
# M4 con GPU (Colab/Kaggle con CUDA, o Apple MPS):
python proy_datasc_tp_mlg.py m4
python proy_datasc_tp_mlg.py m4-pdf --apellido Gomez --nombre MariaLorena
python proy_datasc_tp_mlg.py m5 --apellido Gomez --nombre MariaLorena
```

Orden recomendado: `m2` y `m3` (CPU) → `m4` (GPU) → `m4-pdf` y `m5`, que solo leen los JSON generados por las etapas previas.

C.3 Versiones registradas

Paquete	Versión	Fuente del dato
Python	3.14.6	kernel del notebook M3
numpy	2.5.0	`requirements.txt` generado por el notebook M3
pandas	3.0.3	ídem
scikit-learn	1.9.0	ídem
matplotlib	3.11.0	ídem
spacy	3.8.18 (modelo `en_core_web_sm`)	ídem
torch	2.15.0	`m1_history.json`, `m4_metrics.json`
peft	0.21.2	`adapter_config.json`
transformers, datasets, accelerate, reportlab, joblib	sin versión registrada	fijar con `pip freeze` al entregar

> **Corrección sugerida en el script.** En `main()`, cuando no se pasa la etapa por línea de comandos, el programa pide la etapa **dos veces**: tras validar la primera respuesta ejecuta un segundo `input()` sin validación. Basta con borrar la línea `a.etapa = input("Etapa a ejecutar (m1, m2, m3, m4, m4-pdf, m5): ").strip()` que sigue al `if`

> `a.etapa not in opciones`. Además, `cargar()` imprime las rutas con un `print` de depuración que puede eliminarse. Pasar la etapa como argumento (`python proy_datasc_tp_mlg.py m3`) evita el problema.

C.4 Contenido sugerido de requirements.txt

```
numpy==2.5.0
pandas==3.0.3
scikit-learn==1.9.0
matplotlib==3.11.0
spacy==3.8.18
torch==2.15.0
peft==0.21.2
transformers
datasets
accelerate
reportlab
pillow
joblib
```

Anexo D. Tablas completas y archivos de configuración

D.1 Grilla completa de TF-IDF (validación interna, 18 combinaciones)

#	max_features	ngram_range	Modelo	n_features	F1 macro val.	Ajuste (s)
1	sin límite	"(1	2)"	MultinomialNB	20507	0.896
2	20000	"(1	2)"	MultinomialNB	20000	0.8954
3	sin límite	"(1	2)"	LinearSVC	20507	0.8934
4	20000	"(1	1)"	MultinomialNB	8253	0.8929
5	sin límite	"(1	1)"	MultinomialNB	8253	0.8929
6	20000	"(1	2)"	LinearSVC	20000	0.8921
7	5000	"(1	1)"	MultinomialNB	5000	0.8914
8	5000	"(1	2)"	MultinomialNB	5000	0.8907
9	20000	"(1	2)"	LogisticRegression	20000	0.8891
10	sin límite	"(1	2)"	LogisticRegression	20507	0.8879
11	20000	"(1	1)"	LinearSVC	8253	0.8878
12	sin límite	"(1	1)"	LinearSVC	8253	0.8878
13	20000	"(1	1)"	LogisticRegression	8253	0.883
14	sin límite	"(1	1)"	LogisticRegression	8253	0.883
15	5000	"(1	2)"	LinearSVC	5000	0.8826
16	5000	"(1	2)"	LogisticRegression	5000	0.8804
17	5000	"(1	1)"	LinearSVC	5000	0.8801
18	5000	"(1	1)"	LogisticRegression	5000	0.8785

D.2 Matrices de confusión numéricas (test)

Tabla D2a. A: TF-IDF + MultinomialNB

Real \ Predicho	Business	Sci_Tech	Sports	World
Business	434	38	6	22
Sci_Tech	44	434	7	15
Sports	5	8	484	3
World	34	13	13	440

Tabla D2b. B: DistilBERT + LoRA

Real \ Predicho	Business	Sci_Tech	Sports	World
Business	435	41	3	21
Sci_Tech	49	443	1	7
Sports	6	2	492	0
World	33	10	11	446

D.3 adapter_config.json (resumen)

Campo	Valor
`peft_type` / `task_type`	LORA / SEQ_CLS
`base_model_name_or_path`	distilbert-base-uncased

Campo	Valor
`r` / `lora_alpha` / `lora_dropout`	8 / 16 / 0.1
`target_modules`	q_lin, v_lin
`modules_to_save`	pre_classifier, classifier (y `score`, por defecto de PEFT para SEQ_CLS)
`bias` / `use_dora` / `use_rslora`	none / false / false
`init_lora_weights` / `inference_mode`	true / true
`peft_version`	0.21.2

D.4 Salidas de predicción

Ambos modelos guardan las 2.000 predicciones en el mismo orden del archivo de test (`m3_metrics.json` y `m4_metrics.json`, campo `preds`), lo que habilita los análisis pareados de la sección 5.

Anexo E. Glosario de términos técnicos

Término	Definición
Backpropagation	Algoritmo que calcula el gradiente de la pérdida respecto de cada peso aplicando la regla de la cadena de salida a entrada; PyTorch lo automatiza con *autograd* (`loss.backward()`).
Embedding	Vector denso que representa una palabra o token en un espacio donde la cercanía refleja similitud de uso; en el Transformer es contextual.
Auto-atención (self-attention)	Mecanismo por el que cada token pondera a todos los demás: $\mathrm{softmax}(QK^T/\sqrt{d_k})\cdot V$.
Attention head (cabeza de atención)	Subespacio de atención independiente; DistilBERT tiene 12 por capa, de 64 dimensiones cada una.
TF-IDF	Ponderación de términos: frecuencia en el documento × log(inverso de la frecuencia documental).
Bolsa de palabras (BoW)	Representación que ignora el orden de las palabras y solo cuenta su presencia o frecuencia.
WordPiece	Algoritmo de sub-palabras usado por BERT/DistilBERT; reduce el problema de OOV.
OOV	Token fuera del vocabulario conocido por el modelo.
Fine-tuning	Ajuste de un modelo preentrenado a una tarea concreta.
PEFT	*Parameter-Efficient Fine-Tuning*: familia de técnicas que entrenan una fracción mínima de los parámetros.
LoRA	Adaptación de bajo rango: congela W_0 y aprende $\Delta W = (\alpha/r)\cdot B\cdot A$.
Rango (r)	Dimensión interna de las matrices A y B; controla la capacidad del adaptador.
alpha (α)	Factor de escala de la actualización LoRA (se aplica como α/r).
F1 macro / weighted	Media del F1 por clase sin ponderar / ponderada por soporte; coinciden con clases balanceadas.
Data leakage (fuga de datos)	Uso de información del test durante el entrenamiento o preprocesamiento (p. ej. ajustar el vectorizador con el test).
McNemar	Prueba para comparar dos clasificadores sobre las mismas muestras usando solo los casos en los que discrepan.
Bootstrap	Remuestreo con reemplazo del test para estimar intervalos de confianza.